

WHY DSA: IN MACHINE LEARNING

Let us understand

Let us see Time and Space complexity of ML Algorithms

Rank	Algorithm	Training Time Complexity	Prediction Time Complexity	Space Complexity
1	Naïve Bayes	$O(n \cdot d)$	$O(d)$	$O(n \cdot d)$
2	K-Nearest Neighbors (KNN)	$O(1)$ (lazy learner, no training)	$O(n \cdot d)$	$O(n \cdot d)$
3	Decision Trees	$O(n \cdot \log n)$	$O(\log n)$	$O(n \cdot d)$
4	Logistic Regression	$O(n^2)$ (using gradient descent)	$O(n)$	$O(n \cdot d)$
5	Linear Regression	$O(n^2)$ (matrix inversion)	$O(n)$	$O(n \cdot d)$

- n : Number of data points (samples)
- d : Number of features
- m : Number of models (e.g., trees in Random Forests or boosting rounds)
- k : Number of clusters or discovered rules
- h : Number of hidden units in neural networks
- t : Number of iterations in K-Means

6	K-Means Clustering	$O(n \cdot k \cdot d \cdot t)$	$O(k \cdot d)$	$O(n \cdot k)$
7	Support Vector Machines (SVM)	$O(n^2)$ to $O(n^3)$ (linear kernel) / $O(n^3)$ to $O(n^4)$ (non-linear kernel)	$O(n \cdot d)$	$O(n^2)$
8	Gradient Boosting Machines (GBM)	$O(m \cdot n \cdot \log n)$	$O(m)$	$O(m \cdot n)$
9	AdaBoost	$O(n \cdot m \cdot d)$	$O(m)$	$O(n \cdot m \cdot d)$
10	XGBoost	$O(n \cdot m \cdot \log n)$	$O(m)$	$O(m \cdot n)$

11	LightGBM	$O(n \cdot m \cdot \log n)$	$O(m)$	$O(m \cdot n)$
12	Principal Component Analysis (PCA)	$O(n \cdot d^2)$	$O(d^2)$	$O(n \cdot d)$
13	Deep Learning Models (CNN, RNN, LSTM)	$O(n \cdot d \cdot h^2)$	$O(d \cdot h)$	$O(n \cdot h^2)$
14	Reinforcement Learning (Q-Learning)	$O(n \cdot m \cdot d)$	$O(1)$	$O(m \cdot d)$
15	Random Forests	$O(m \cdot n \cdot \log n)$	$O(m \cdot \log n)$	$O(m \cdot n)$

1. Linear Regression

- Training: $O(n^2)$ (for solving using matrix inversion, assuming n features)
- Prediction: $O(n)$ (for n features)

2. Logistic Regression

- Training: $O(n^2)$ (using gradient descent, can depend on the number of iterations)
- Prediction: $O(n)$

3. K-Nearest Neighbors (KNN)

- Training: $O(1)$ (KNN is a lazy learner, it does not have a training phase)
- Prediction: $O(n \cdot d)$, where n is the number of data points and d is the number of dimensions (features)

4. Decision Trees

- Training: $O(n \cdot \log n)$, where n is the number of samples (can vary based on the implementation and tree depth)
- Prediction: $O(\log n)$, due to the depth of the tree

5. Random Forests

- Training: $O(m \cdot n \cdot \log n)$, where m is the number of trees and n is the number of samples
- Prediction: $O(m \cdot \log n)$

6. Support Vector Machines (SVM)

- Training:
 - Linear kernel: $O(n^2)$ to $O(n^3)$
 - Non-linear kernel: $O(n^3)$ to $O(n^4)$, where n is the number of samples
- Prediction: $O(n \cdot d)$, where n is the number of support vectors and d is the number of features

7. Naïve Bayes

- Training: $O(n \cdot d)$, where n is the number of samples and d is the number of features
- Prediction: $O(d)$, where d is the number of features

8. K-Means Clustering

- Training: $O(n \cdot k \cdot d \cdot t)$, where n is the number of data points, k is the number of clusters, d is the number of features, and t is the number of iterations
- Prediction: $O(k \cdot d)$

9. Principal Component Analysis (PCA)

- Training: $O(n \cdot d^2)$, where n is the number of samples and d is the number of features (for the covariance matrix eigenvalue decomposition)
- Prediction: $O(d^2)$, after dimensionality reduction

10. Neural Networks (Feedforward)

- Training: $O(n \cdot d \cdot h)$, where n is the number of samples, d is the number of features, and h is the number of hidden units
- Prediction: $O(d \cdot h)$

11. Gradient Boosting Machines (GBM) / XGBoost

- Training: $O(m \cdot n \cdot \log n)$, where m is the number of trees and n is the number of samples
- Prediction: $O(m)$

12. AdaBoost

- Training: $O(n \cdot m \cdot d)$, where m is the number of estimators (weak learners), n is the number of samples, and d is the number of features
- Prediction: $O(m)$

13. XGBoost (Extreme Gradient Boosting)

- Training: $O(n \cdot m \cdot \log n)$, where m is the number of boosting rounds and n is the number of samples
- Prediction: $O(m)$

14. LightGBM (Light Gradient Boosting Machine)

- Training: Similar to XGBoost, but optimized for speed and memory, roughly $O(n \cdot m \cdot \log n)$
- Prediction: $O(m)$

15. Deep Learning Models (CNN, RNN, LSTM)

- Training: $O(n \cdot d \cdot h^2)$ (highly dependent on architecture, number of layers, and parameters)
- Prediction: $O(d \cdot h)$ (again, dependent on the architecture)

16. Reinforcement Learning (Q-Learning)

- Training: $O(n \cdot m \cdot d)$, where n is the number of episodes, m is the number of states, and d is the number of actions
- Prediction: $O(1)$ (for choosing an action based on the policy)

17. Association Rule Learning (Apriori)

- Training: $O(n^2 \cdot d)$, where n is the number of transactions and d is the number of items
- Prediction: $O(k)$, where k is the number of discovered rules

Ranking Criteria:

- **Rank 1–5:** Algorithms that are highly efficient in both time and space complexity, often well-suited for large datasets.
- **Rank 6–10:** Algorithms that are efficient in one area (e.g., space or time) but may require significant resources in others.
- **Rank 11–15:** Algorithms that can be resource-intensive, particularly with deep learning models, large ensembles, or complex kernels.
- This ranking aims to strike a balance between **training time**, **prediction time**, and **space usage**. More efficient algorithms typically have better ranks, while those with higher complexities, especially deep learning or kernel-based methods, rank lower.

So we are starting DSA
with very basic idea about
ARRAY

Introduction to Arrays

➤ Definition:

- An array is a **collection of elements** stored in a **contiguous memory location**, and elements are accessed using **indices**.

➤ Characteristics:

- Fixed size (in most languages; Python allows dynamic resizing with lists)
- Homogeneous data (in static-typed languages; Python lists are flexible)
- Supports random access: $O(1)$ time to access any element

Python Example:

python

 Copy

 Edit

```
# Creating an array using list
```

```
arr = [10, 20, 30, 40, 50]
```

```
# Accessing elements
```

```
print(arr[0]) # Output: 10
```

```
print(arr[3]) # Output: 40
```

Declaration and Initialization

1D Array:

python

 Copy

 Edit

```
# Declaring and initializing  
arr = [1, 2, 3, 4, 5]
```

2D Array:

python

 Copy

 Edit

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
print(matrix[1][2]) # Output: 6
```

Initialization with zeros:

python

 Copy

 Edit

```
zeros = [0] * 5 # [0, 0, 0, 0, 0]
```

Basic Operations on Arrays

Traversal

python

 Copy

 Edit

```
arr = [10, 20, 30, 40]
```

```
# Using loop
```

```
for i in arr:
```

```
    print(i)
```

Insertion

python

 Copy

 Edit

```
arr = [10, 20, 30]
```

```
arr.insert(1, 15) # Insert 15 at index 1
```

```
print(arr) # [10, 15, 20, 30]
```

— Deletion

python

 Copy

 Edit

```
arr = [10, 20, 30, 40]
arr.remove(30)      # Remove value
del arr[1]          # Remove by index
```

Searching

python

 Copy

 Edit

```
arr = [10, 20, 30, 40]

# Linear search
if 30 in arr:
    print("Found")
```

Time and Space Complexity

✓ Access:

- Accessing an element by index: $O(1)$

✓ Search:

- Linear Search: $O(n)$
- Binary Search (sorted array): $O(\log n)$

✓ Insertion:

- At end (amortized): $O(1)$
- At middle/start: $O(n)$ (due to shifting)

✓ Deletion:

- By value or index: $O(n)$ (worst case)

✓ Space:

- Array of size n takes $O(n)$ space

Common Array Problems (with Python Code)

Reverse an Array

python

 Copy

 Edit

```
arr = [1, 2, 3, 4, 5]
arr.reverse()
print(arr) # [5, 4, 3, 2, 1]
```

Move Zeroes to End

python

 Copy

 Edit

```
arr = [0, 1, 0, 3, 12]
result = [x for x in arr if x != 0] + [0] * arr.count(0)
print(result) # [1, 3, 12, 0, 0]
```

Kadane's Algorithm (Max Subarray Sum)

python

 Copy

 Edit

```
arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
max_sum = curr = arr[0]

for num in arr[1:]:
    curr = max(num, curr + num)
    max_sum = max(max_sum, curr)

print(max_sum) # Output: 6
```

Rotate Array (by 1)

python

 Copy

 Edit

```
arr = [1, 2, 3, 4, 5]
rotated = arr[1:] + arr[:1]
print(rotated) # [2, 3, 4, 5, 1]
```

Advanced Concepts

☐ Prefix Sum

python

 Copy

 Edit

```
arr = [1, 2, 3, 4]
prefix = [0] * len(arr)
prefix[0] = arr[0]

for i in range(1, len(arr)):
    prefix[i] = prefix[i - 1] + arr[i]

print(prefix) # [1, 3, 6, 10]
```

Sliding Window (Max Sum of Subarray Size k)

python

 Copy

 Edit

```
arr = [1, 4, 2, 10, 23, 3, 1, 0, 20]
k = 4
max_sum = sum(arr[:k])
window_sum = max_sum

for i in range(len(arr) - k):
    window_sum = window_sum - arr[i] + arr[i + k]
    max_sum = max(max_sum, window_sum)

print(max_sum)  # Output: 39
```

Practice Interview Questions

- ☐ Remove duplicates from sorted array
- ☐ Two Sum using HashMap
- ☐ Merge two sorted arrays
- ☐ Maximum product subarray
- ☐ Leaders in an array
- ☐ Stock buy-sell for max profit

Soon will share talk about Sorting.....